

Deep Dive — Module 4: Spring Boot (Java Backend)

***Goal:** understand the central orchestrator — REST controllers, services, repositories, entities, how it calls the Python embedding service and Qdrant, security with JWT, configuration, and global error handling. Every other component connects through Spring Boot.*

Where Spring Boot sits: It's the **brain and traffic controller**. The React UI calls it; it parses files, chunks text, calls the embedding service, upserts to Qdrant, persists to PostgreSQL, enforces security, and writes audit logs. If you understand this module, you understand how the whole system is wired.

1. REST API basics

A **REST API** is a set of URLs (endpoints) the frontend calls with HTTP methods. Each does one thing:

POST	/api/documents/upload	→ index a file, return document id
POST	/api/documents/search	→ return similar documents
GET	/api/documents	→ paginated list
GET	/api/documents/{id}	→ one document's details
DELETE	/api/documents/{id}	→ delete (admin; GDPR erasure)
GET	/api/alerts	→ active fraud/duplicate alerts
PATCH	/api/alerts/{id}	→ acknowledge/dismiss
GET	/api/health	→ service health

HTTP **status codes** carry meaning: 200 OK, 202 Accepted (queued), 400 bad input, 401 unauthenticated, 403 forbidden, 404 not found, 422 unprocessable, 429 rate-limited, 500/502 server/upstream errors.

2. The layered architecture

Requests flow through clearly separated layers — this separation is what keeps the code testable and maintainable:

```
Controller → receives HTTP, validates input, returns response (thin)
  ↓
Service → business logic / orchestration (thick)
  ↓
Repository → database access (Spring Data JPA)
Entity → Java class mapped to a DB table
```

Plus cross-cutting pieces: parsers/OCR, the embedding HTTP client, the Qdrant client, security filters, and config.

3. Controllers

A `@RestController` maps HTTP requests to Java methods.

```
@RestController
@RequestMapping("/api/documents")
public class DocumentController {

    @Autowired private DocumentService documentService;

    @PostMapping("/upload")
    public ResponseEntity<UploadResponseDTO> upload(@RequestParam("file") MultipartFile file) {
        return ResponseEntity.accepted().body(documentService.handleUpload(file)); // 202
    }

    @GetMapping("/{id}")
    public ResponseEntity<DocumentDTO> getById(@PathVariable UUID id) {
        return ResponseEntity.ok(documentService.findById(id)); // 200
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')") // RBAC at the method level
    public ResponseEntity<Void> delete(@PathVariable UUID id) {
        documentService.delete(id);
        return ResponseEntity.ok().build();
    }
}
```

- `@PathVariable` extracts `{id}`; `@RequestParam` reads form fields/files; `@RequestBody` binds JSON.
- Keep controllers **thin** — validate and delegate, don't put logic here.

4. Services (business logic)

The `@Service` layer orchestrates the work. The upload pipeline lives here:

```
@Service
public class DocumentService {
    @Autowired private ChunkingService chunkingService;
    @Autowired private EmbeddingService embeddingService;
    @Autowired private QdrantService qdrantService;
    @Autowired private DocumentRepository documentRepository;
    @Autowired private AuditService auditService;

    @Transactional
    public UploadResponseDTO handleUpload(MultipartFile file) {
        String text = extractText(file); // PDFBox/POI
        if (text.trim().isEmpty()) text = ocrService.extractText(file); // OCR fallback
        List<String> chunks = chunkingService.chunk(text); // ~800 chars each
        List<float[]> vectors = embeddingService.getEmbeddings(chunks); // → Python
        qdrantService.upsert(chunks, vectors, metadata); // → Qdrant
        Document saved = documentRepository.save(new Document(file, metadata)); // → Postgres
        auditService.log("DOCUMENT_UPLOAD", saved.getId()); // audit
        return new UploadResponseDTO(saved.getId(), "INDEXED");
    }
}
```

`@Transactional` wraps the PostgreSQL writes in one transaction (Module 1 §11): the document row and audit row commit together or not at all.

Dependency Injection: `@Autowired` (or constructor injection, preferred) lets Spring supply the collaborators. You depend on interfaces, which makes mocking trivial in unit tests.

5. Repositories (Spring Data JPA)

Define an interface; Spring generates the SQL from the method name.

```
public interface DocumentRepository extends JpaRepository<Document, UUID> {  
    List<Document> findByVendor(String vendor); // WHERE vendor = ?  
    List<Document> findByDeletedAtIsNull(); // active only  
    List<Document> findByVendorAndDeletedAtIsNull(String vendor);  
  
    @Query("SELECT d FROM Document d WHERE d.totalAmount > :amount AND d.deletedAt IS NULL")  
    List<Document> findExpensive(@Param("amount") BigDecimal amount); // custom JPQL  
}
```

`JpaRepository` gives `save`, `findById`, `findAll(Pageable)`, `delete`, etc. for free.

Pagination (`Pageable`) backs the `GET /documents` endpoint. Queries are parameterized → no SQL injection.

6. Entities (object ↔ table mapping)

An `@Entity` maps a Java class to a table; fields map to columns.

```
@Entity @Table(name = "documents")  
public class Document {  
    @Id @GeneratedValue private UUID id;  
    @Column(nullable = false) private String filename;  
    private String vendor;  
    @Column(name = "total_amount") private BigDecimal totalAmount; // money → BigDecimal  
    @Column(name = "uploaded_at", nullable = false) private LocalDateTime uploadedAt = LocalDateTime.now();  
    @Column(name = "deleted_at") private LocalDateTime deletedAt; // null = active (soft delete)  
  
    @ManyToOne @JoinColumn(name = "uploaded_by")  
    private User uploadedBy; // FK relationship  
}
```

- `BigDecimal` mirrors PostgreSQL `NUMERIC` for exact money (never `double`).
 - `@ManyToOne` models "many documents belong to one user."
 - **DTO vs Entity:** controllers return DTOs (e.g. `DocumentDTO`), not entities, so internal fields and lazy relationships don't leak into the API.
-

7. Calling the Python embedding service

Spring calls the Flask service over HTTP using `RestTemplate` (simple) or `WebClient` (non-blocking, preferred for many parallel chunk calls).

```
@Service
public class EmbeddingService {
    private final RestTemplate restTemplate = new RestTemplate();
    private final String url = "http://localhost:5000/embed";

    public float[] getEmbedding(String text) {
        Map<?,?> res = restTemplate.postForObject(url, Map.of("text", text), Map.class);
        List<Double> list = (List<Double>) res.get("embedding");
        float[] v = new float[list.size()];
        for (int i = 0; i < list.size(); i++) v[i] = list.get(i).floatValue();
        return v; // 384 floats
    }
}
```

For multiple chunks, prefer the `/embed-batch` endpoint to cut HTTP round-trips. Wrap calls with timeouts and a fallback so an embedding-service outage returns a clean `502 UPSTREAM_ERROR` rather than hanging.

8. Spring Security + JWT

A filter validates the JWT on **every** request before the controller runs (deep dive in Module 8).

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Override protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) throws ServletException, IOException {
        String token = extractToken(req);
        if (token != null && jwtService.isValid(token)) {
            String role = jwtService.extractRole(token);
            SecurityContextHolder.getContext().setAuthentication(new JwtAuthentication(token, role));
        }
        chain.doFilter(req, res); // continue to controller
    }
}
```

Once the security context holds the role, `@PreAuthorize("hasRole('ADMIN')")` and `hasAnyRole(.)` enforce RBAC at the method level. Department-level filtering (clerks see

only their department) is applied in the service layer.

9. Configuration (`application.yml`)

Externalized config — no secrets hard-coded; use env vars / a vault.

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/financial_docs
    username: app_user
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: validate          # verify entities match the schema; never 'update' in prod
server:
  port: 8080
  servlet:
    multipart:
      max-file-size: 50MB        # reject oversized uploads
embedding: { service: { url: http://localhost:5000 } }
qdrant:    { host: localhost, port: 6333, collection: financial_documents }
jwt:       { secret: ${JWT_SECRET}, expiry-hours: 24 }
```

`ddl-auto: validate` is the safe choice — schema changes go through migrations (e.g. Flyway/Liquibase), not Hibernate auto-DDL.

10. Global error handling

A single `@RestControllerAdvice` converts exceptions into the project's consistent error shape — and never leaks stack traces.

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(DocumentNotFoundException.class)
    public ResponseEntity<ErrorResponse> notFound(DocumentNotFoundException ex) {
        return ResponseEntity.status(404).body(new ErrorResponse(404, "RESOURCE_NOT_FOUND", ex.getMessage()));
    }
    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> forbidden(AccessDeniedException ex) {
        return ResponseEntity.status(403).body(new ErrorResponse(403, "ACCESS_DENIED", "Insufficient permissions"));
    }
}
// { "status":404, "errorCode":"RESOURCE_NOT_FOUND", "message":"...", "timestamp":"..." }

```

This maps to the project's error taxonomy: `VALIDATION_ERROR` (400), `AUTH_FAILED` (401), `ACCESS_DENIED` (403), `EXTRACTION_FAILED` (422), `RATE_LIMIT_EXCEEDED` (429), `UPSTREAM_ERROR` (502), `INTERNAL_ERROR` (500).

11. Cross-cutting concerns

- **Validation:** Bean Validation (`@NotNull` , `@Size` , `@Pattern`) on DTOs rejects bad input early.
- **Rate limiting:** a filter (in-memory sliding window / Bucket4j) caps login, upload, and search rates per the security spec.
- **Logging:** SLF4J + Logback in structured JSON, with `requestId` , `userId` , `action` , `durationMs` , `outcome` .
- **Metrics:** Actuator + Micrometer expose metrics for Prometheus/Grafana; `/health` reports Qdrant/Postgres/embedding status.
- **Package structure:** `controller` , `service` , `parser` , `ocr` , `embedding` , `vector` , `repository` , `model` , `security` , `config` .

12. Common pitfalls

- **Fat controllers** — logic belongs in services.
- **Returning entities from the API** — leaks internals and triggers lazy-loading errors; return DTOs.

- **double for money** — use `BigDecimal` / `NUMERIC` .
 - **ddl-auto: update in production** — silent, unsafe schema changes; use migrations + `validate` .
 - **No timeout on the embedding call** — a hung Python service hangs the request thread; set timeouts and map to `502` .
 - **Forgetting `@Transactional`** — partial writes (document without audit row).
 - **Field injection everywhere** — prefer constructor injection for testability and immutability.
 - **Secrets in `application.yml` / `images`** — use env vars / vault.
-

13. Practice exercises

`spring init--dependencies=web,data-jpa,security,postgresql demo` to scaffold.

1. Create a `DocumentController` with `upload` , `getById` , and `delete` endpoints returning the right status codes.
 2. Add a `DocumentService.handleUpload` that calls (mocked) chunking, embedding, Qdrant, repository, and audit collaborators in order.
 3. Define `DocumentRepository` with `findByDeletedAtIsNull()` and a paginated `findAll(Pageable)` ; back a `GET /documents` endpoint.
 4. Map a `Document` entity to the `documents` table with `BigDecimal totalAmount` and a `@ManyToOne` to `User` .
 5. Write `EmbeddingService.getEmbedding` calling a local Flask `/embed` ; add a 2-second timeout and a fallback that throws an upstream exception.
 6. Add `@PreAuthorize` so only `ADMIN` can `delete` and only `FINANCE_MANAGER` / `ADMIN` can read alerts; test with different roles.
 7. Add a `@RestControllerAdvice` mapping a custom `DocumentNotFoundException` to a 404 in the project's error shape.
 8. Wrap the document + audit insert in `@Transactional` ; force the second insert to fail and confirm rollback.
-

14. Self-check questions

- What does each layer (controller/service/repository/entity) do, and why separate them?
 - How does Spring Data JPA turn `findByVendorAndDeletedAtIsNull` into SQL?
 - Why return DTOs instead of entities from controllers?
 - Where and how is RBAC enforced?
 - What does `@Transactional` guarantee in the upload flow?
 - Why `ddl-auto: validate` instead of `update` ?
 - How should the backend behave if the embedding service is down?
 - Why is `BigDecimal` used for amounts?
-

15. Glossary

- **REST endpoint** — a URL + HTTP method doing one operation.
 - **Controller / Service / Repository / Entity** — the four layers.
 - **DTO** — Data Transfer Object; the API-facing shape.
 - **Dependency Injection** — Spring supplies collaborators (`@Autowired` /constructor).
 - **JPA / Hibernate** — object-relational mapping; objects ↔ tables.
 - **JPQL** — JPA query language over entities.
 - `@Transactional` — wraps a method in a DB transaction.
 - `@PreAuthorize` — method-level role check.
 - **Actuator** — Spring's ops endpoints (health/metrics).
-

Navigation: [← Module 3 React](#) | [Index](#) | [Module 5 Python/Embeddings](#) →